

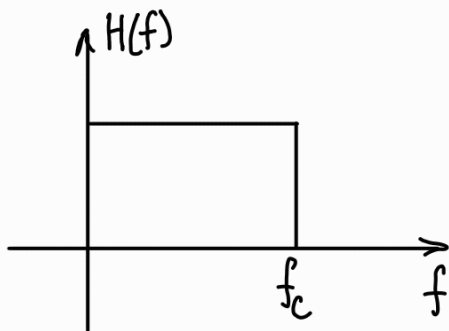
At the end of the last lecture we made 2 claims —

- ① We will let our filters develop (learn) through some iterative process
- ② We will cascade filters to create better filter

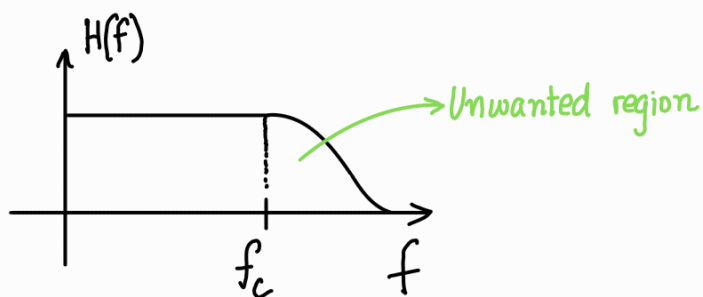
The 1st point will take us through backpropagation and the 2nd point will be the foundation for modern CNN networks. Let us tackle point 2 first. The learning part will be covered eventually through a mix of lab and theory.

Filter order & deviation from ideal response —

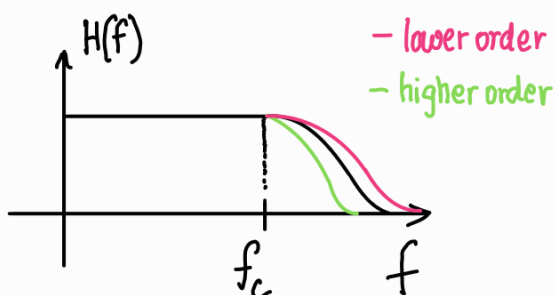
An ideal LPF frequency response —



A practical LPF frequency response —



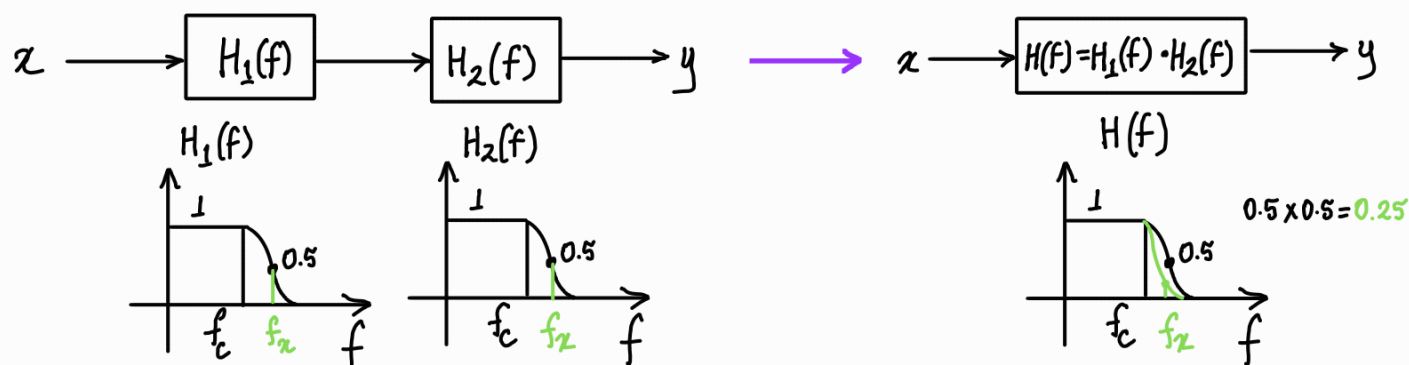
The area of the unwanted region can be reduced by increasing the filter order —



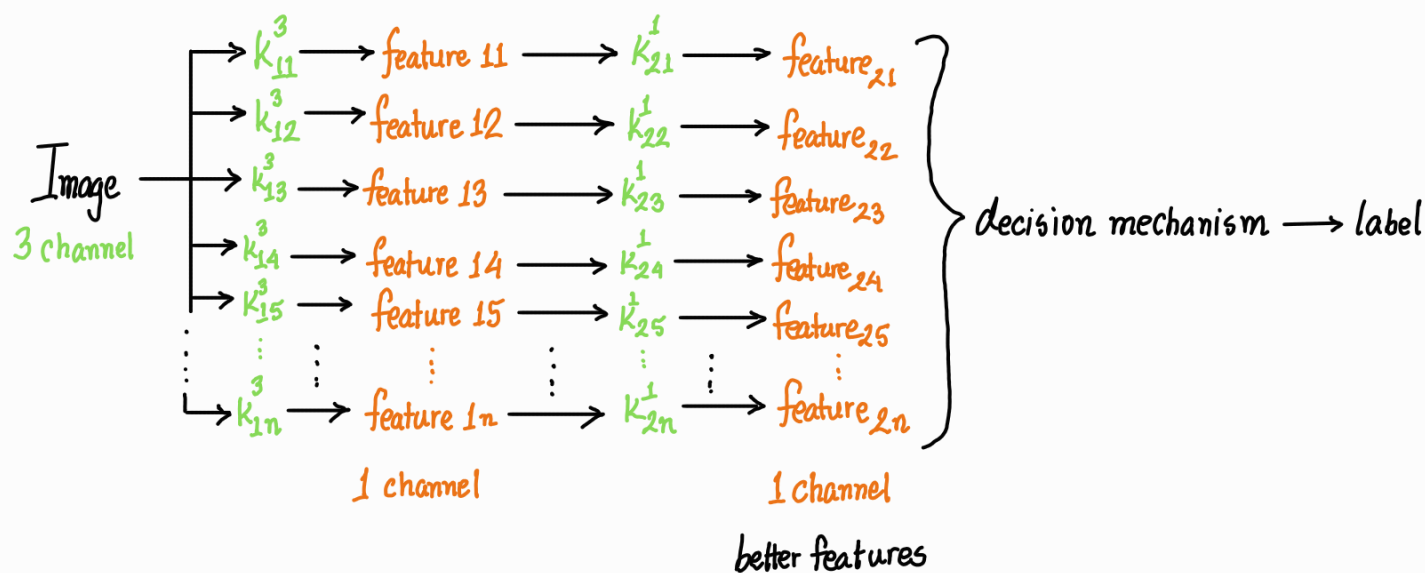
Convolutional filters are FIR (Finite Impulse Response) filters where order, $N = K - 1$
↪ length of kernel

So small kernels are not that great.

Can we improve how good our filter can do? What if we cascade 2 practical LPF in series?



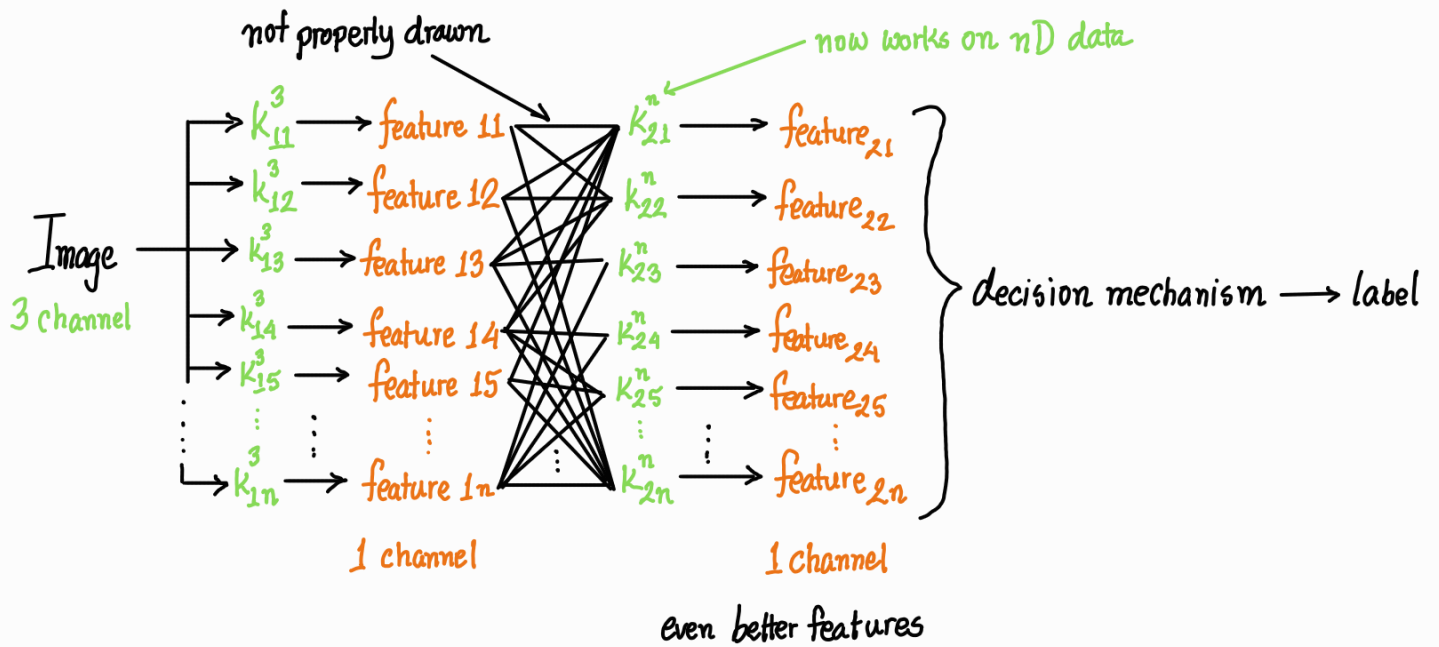
So it actually does work. So now let's use this concept to improve our previous cat and dog classifier system. We will just improve the feature extraction part —



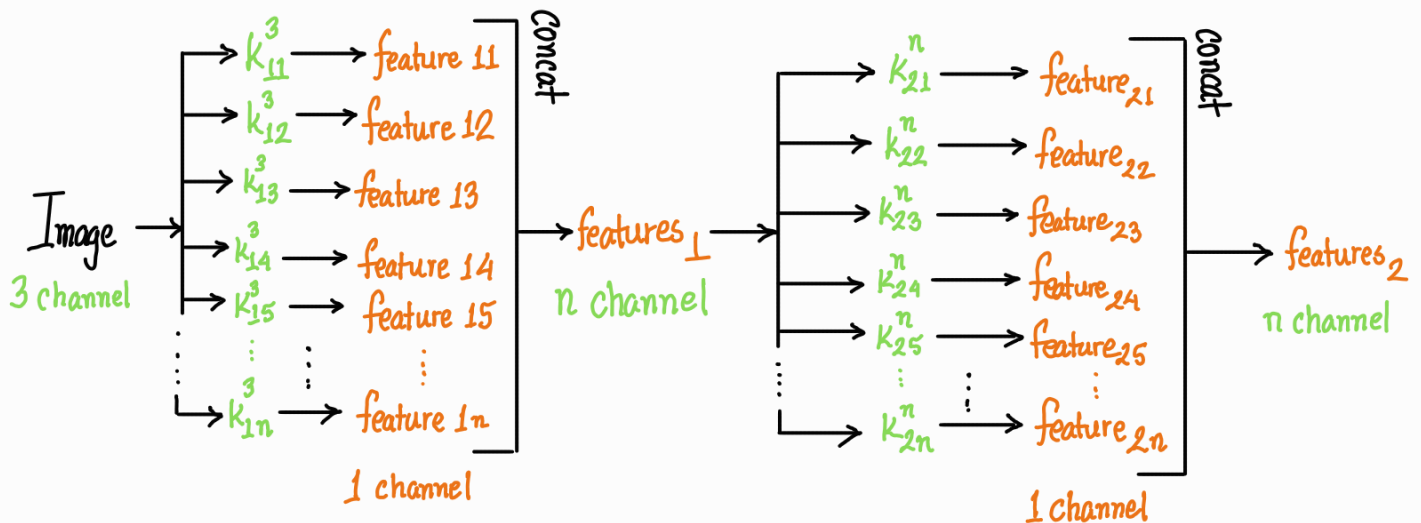
Assume all kernels and features of a level to be of same dims.

Let's think of some improvements. Since our goal is to filter the input signal or image, and not individual intermediate features, it's better to not segregate the feature paths unless we are stumped by the computational weight. It helps to create better feature extraction filters for the original signal.

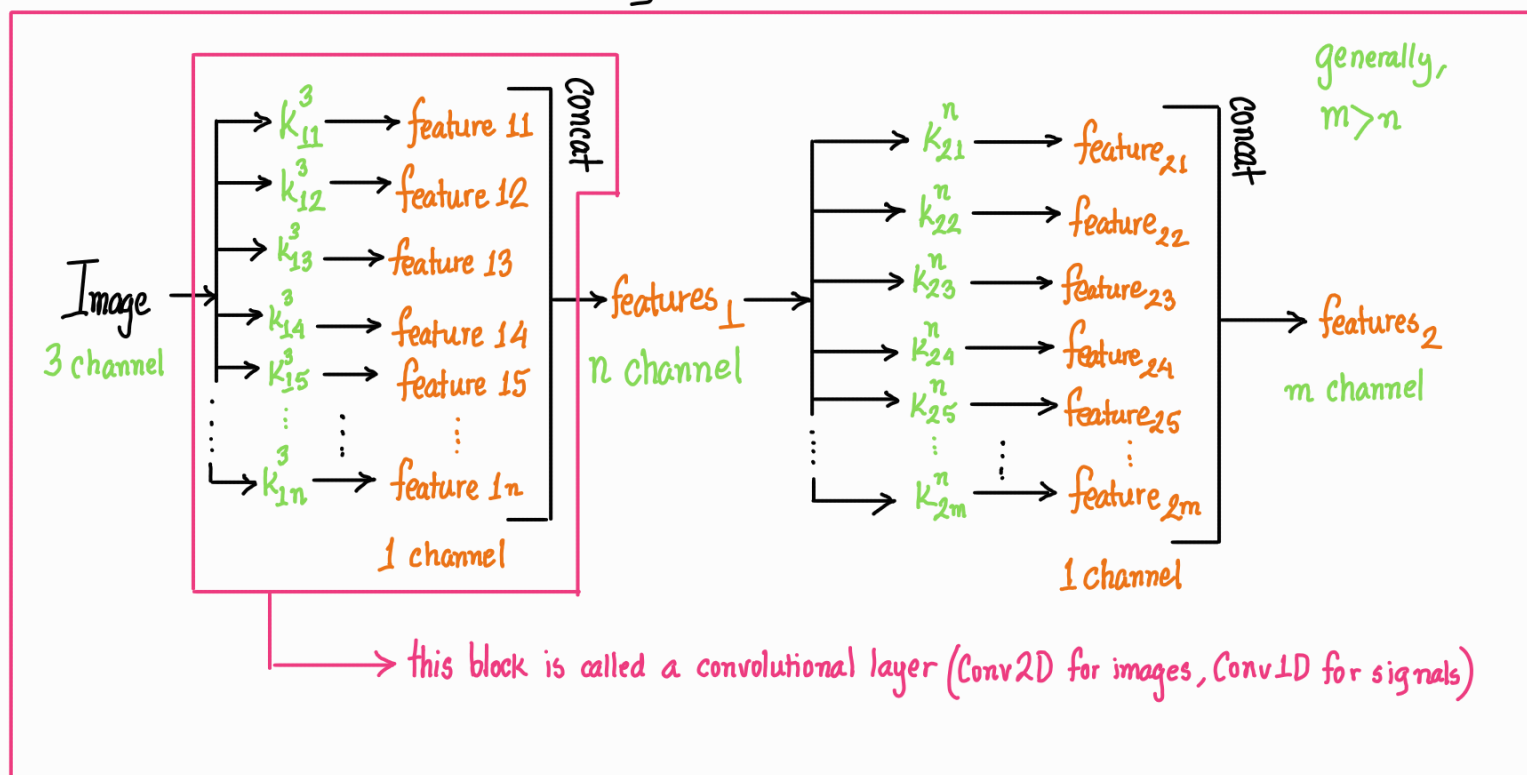
So the system becomes —



We can represent the system more compactly —

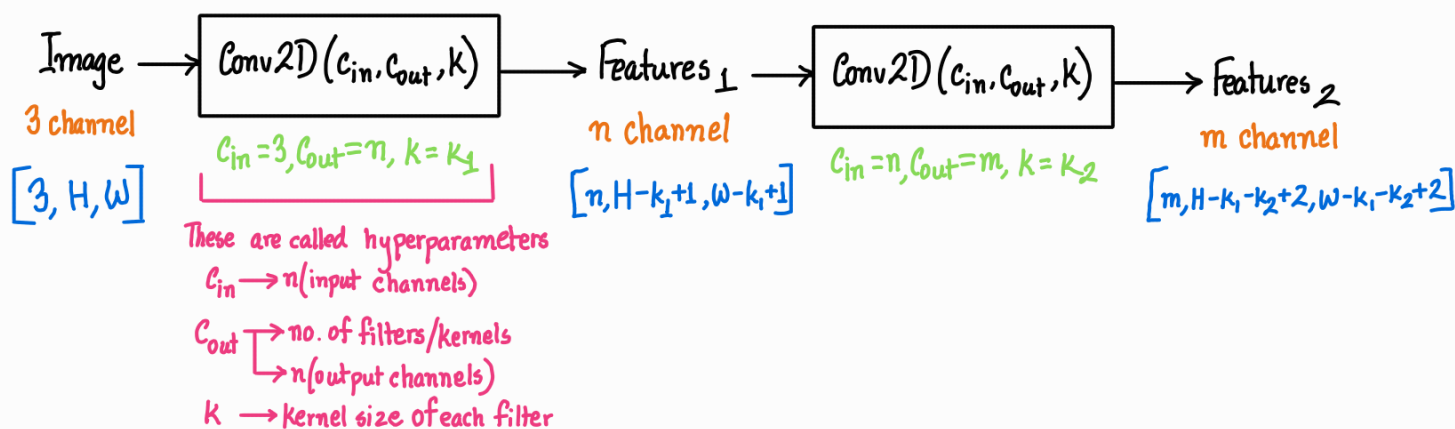


The early features are kind of generalized and kind of redundant. So we generally keep the feature dims low at first and gradually increase it over the layers —

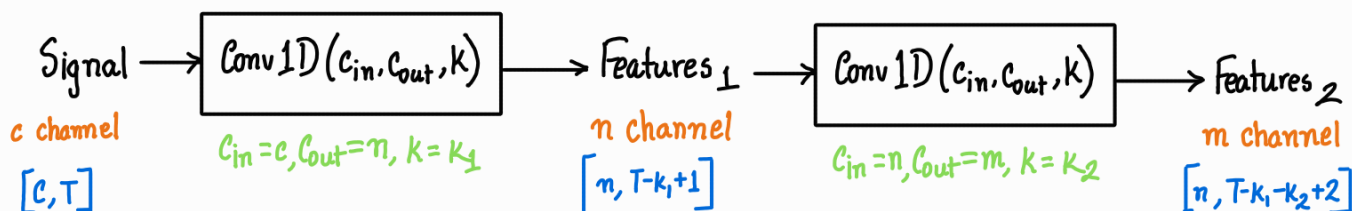


This is a simple convolutional neural network (CNN) backbone

So we can represent this CNN more compactly —



A sample CNN for 1D signals would look like —



Major caveat, minor implication — CNN uses cross correlation & not convolution!!

Lets talk a bit about cross correlation filtering at first —

Although cross correlation and convolution are very different in terms of motivation. The mathematical model is kind of close. Cross correlation based filters are often used in various applications even though they are not theoretically backed. For cross correlation filters, the filter sequence isn't reversed. Everything else is the same as convolutional filters. So technically if a filter is symmetric about the independent axes, both convolutional filtering and cross correlation filtering will give the same output.

1D symmetric filters —

0.2	0.2	0.2	0.2	0.2
-----	-----	-----	-----	-----

x plane of symmetry

1	2	3	2	1
---	---	---	---	---

1D asymmetric filters —

-1	-2	0	1	1
----	----	---	---	---

asymmetric

1	2	3	4	5
---	---	---	---	---

x

2D symmetric filters —

1	2	1
2	1	2
1	2	1

y axis symmetry plane

x axis symmetry plane

2D asymmetric filters —

-1	0	1
-1	0	1
-1	0	1

x

Now CNNs learn the filters from scratch. So it doesn't matter if the filters are reversed or not. It can directly learn $h(-x)$ or $h(-x, -y)$. So we don't bother reversing the filters and save ourselves some computational overhead.

Where to go from here?

We still need to complete our cat and dog classifier. We have developed the feature extractor. But we still need to develop the decision mechanism. For that you need concepts of linear layers. You will learn about linear layers and learning systems in the next lab. We will pick up on the classifier again in the next theory class.